Pass an email address like SPLIT_PART(email_address, '@', 1) to return the first part before the @ symbol. The first part has a position of 1 so 1 was provided as the third argument.

Run the SPLIT_PART() example below in psql to get familiar with the function.

sql/select_split_part_function.sql

```
SELECT split_part('bob@example.com', '@', 1);
--  split_part
--  ------------
--   bob
```

Calling SPLIT_PART() returns bob which is visible in the commented out portion. bob is the text you'll replace with your function.

How long is bob? Use the built-in LENGTH() function. bob has a length of 3.

sql/select_length_function.sql

```
SELECT length('bob');
--  length
-- --------
--       3
```

This is the basic process to build on top of built-in functions as needed.

Next you'll need some replacement text.

Use the built-in PostgreSQL MD5 hash string generation functions. The MD5() function generates 32-character strings from an input string. You could use the text you want to hash as input but MD5 hashed text is not considered secure cryptographically. The original text could be guessed by comparing hashed text with it.

For the Performance database, it's OK to lose the original values. Use the RANDOM() function as input to MD5() but coerce the input to ::text. That provides a semi random input to the MD5() function that isn't connected to the original source text.

The MD5() function generates long strings but you want to take a slice out that matches the length of your source email address, which is typically going to be less than 32 characters for the part before the "@" symbol.

One gotcha when generating MD5 text strings and comparing the first few characters is that there can be overlaps on those first few characters. You'll want to store unique values for the email address, so use a workaround.

The workaround is to make the generated replacement at least 5 characters long. Use the built-in GREATEST() function to use either the length of the original email address input text or a length of 5, whichever is greater.